

Esercizio 2 (8 punti) Data una stringa di numeri interi $A = (a_1, a_2, \dots, a_n)$, si consideri la seguente ricorrenza $z(i, j)$ definita per ogni coppia di valori (i, j) con $1 \leq i, j \leq n$:

$$z(i, j) = \begin{cases} a_j & \text{if } i = 1, 1 \leq j \leq n, \\ a_{n+1-i} & \text{if } j = n, 1 < i \leq n, \\ (z(i-1, j) \cdot z(i, j+1)) - z(i-1, j+1) & \text{altrimenti.} \end{cases}$$

1. Si fornisca il codice di un algoritmo iterativo bottom-up `COMPUTE_Z(A)` che, data in input la stringa A restituisca in uscita il valore $z(n, 1)$.
2. Si valuti il numero esatto $T_{CZ}(n)$ di operazioni tra interi eseguite dall'algoritmo sviluppato al punto (a).

Soluzione:

1. Date le dipendenze tra gli indici nella ricorrenza, un modo corretto di riempire la tabella è attraverso una scansione “reverse column-major”, in cui calcoliamo gli elementi della tabella in ordine decrescente di indice di colonna e, all'interno della stessa colonna, in ordine crescente di indice di riga. Il codice è il seguente.

```
COMPUTE_Z(A)
n = length(A)
for i=1 to n do
    z[1,i] = a_i
    z[i,n] = a_{n+1-i}
for j=n-1 downto 1 do
    for i=2 to n do
        z[i,j] = (z[i-1,j] * z[i,j+1]) - z[i-1,j+1]
return z[n,1]
```

Si osservi che un altro modo corretto di riempire la tabella è attraverso una scansione “reverse diagonal”, che scansiona per diagonal parallele alla diagonale principale partendo da quella contenente solo $z[1, n]$.

2. Ogni iterazione del doppio ciclo dell'algoritmo esegue due operazioni tra interi, e quindi

$$\begin{aligned} T_{CZ}(n) &= \sum_{j=1}^{n-1} \sum_{i=2}^n 2 \\ &= \sum_{j=1}^{n-1} 2(n-1) \\ &= 2(n-1)^2. \end{aligned}$$

Esercizio 2 (8 punti) Per $n > 0$, siano dati due vettori a componenti intere $\mathbf{a}, \mathbf{b} \in \mathbf{Z}^n$. Si consideri la quantità $c(i, j)$, con $0 \leq i \leq j \leq n-1$, definita come segue:

$$c(i, j) = \begin{cases} a_i & \text{se } 0 < i \leq n-1 \text{ e } j = n-1, \\ b_j & \text{se } i = 0 \text{ e } 0 \leq j \leq n-1, \\ c(i-1, j) \cdot c(i, j+1) & 0 < i \leq j < n-1. \end{cases}$$

Si vuole calcolare la quantità $M = \max\{c(i, j) : 0 \leq i \leq j \leq n-1\}$.

1. Si fornisca il codice di un algoritmo iterativo bottom-up per il calcolo di M .
2. Si valuti la complessità esatta dell'algoritmo, associando costo unitario ai prodotti tra numeri interi e costo nullo a tutte le altre operazioni.

Soluzione:

1. Date le dipendenze tra gli indici nella ricorrenza, un modo corretto di riempire la tabella è attraverso una scansione in cui calcoliamo gli elementi in ordine crescente di indice di riga e, per ogni riga, in ordine decrescente di indice di colonna. Il codice è il seguente.

```

COMPUTE(a,b)
n <- length(a)
M = -infinito
for i=1 to n-1 do
  C[i,n-1] <- a_i
  M <- MAX(M,C[i,n-1])
for j=0 to n-1 do
  C[0,j] <- b_j
  M <- MAX(M,C[0,j])
for i=1 to n-2 do
  for j=n-2 downto i do
    C[i,j] <- C[i-1,j] * C[i,j+1]
    M <- MAX(M,C[i,j])
return M

```

2.

$$T(n) = \sum_{i=1}^{n-2} \sum_{j=i}^{n-2} 1 = \sum_{i=1}^{n-2} n-1-i = \sum_{k=1}^{n-2} k = (n-1)(n-2)/2.$$

Esercizio 2 (10 punti) Si ricordi che data una sequenza $X = x_1 \dots x_k$, si indica con X_i il prefisso $x_1 \dots x_i$. Una sottosequenza di X è $x_{i_1} \dots x_{i_h}$ con $1 \leq i_1 < i_2 < \dots < i_h \leq k$, ovvero è una sequenza ottenuta da X eliminando alcuni elementi. Quando Y è sottosequenza di X si scrive $Y \sqsubseteq X$.

Realizzare un algoritmo che, date due sequenze $X = x_1 \dots x_k$ e $Y = y_1 \dots y_h$ determina una *shortest common supersequence* (SCS) ovvero una sequenza Z , di lunghezza minima, tale che $X \sqsubseteq Z$ e $Y \sqsubseteq Z$. Ad esempio per $X = abf$ e $Y = afgj$ una SCS è $abfgj$.

- i. Dare una caratterizzazione ricorsiva della lunghezza $l_{i,j}$ di una SCS di X_i e Y_j e dedurne un algoritmo;
- ii. valutare la complessità dell'algoritmo.

Soluzione: La caratterizzazione ricorsiva è:

$$l_{i,j} = \begin{cases} i + j & \text{if } i = 0 \text{ or } j = 0 \\ l_{i-1,j-1} + 1 & \text{if } i, j > 0 \text{ e } x_i = y_j \\ \min\{l_{i,j-1}, l_{i-1,j}\} + 1 & \text{if } i, j > 0 \text{ e } x_i \neq y_j \end{cases}$$

Ne segue l'algoritmo che riceve in input le stringhe, nella forma di array di caratteri $X[1, k]$, $Y[1, h]$ e usa una matrice $L[0..k, 0..h]$ dove $L[i, j]$ rappresenta la lunghezza della minima SCS di X_i e Y_j .

```
SCSleng (X,Y,k,h)
  for i=0 to k
    L[i,0] = i
  for j=1 to h
    L[0,j] = j

  for i=1 to k
    for j = 1 to h
      if (X[i] = Y[j])
        L[i,j] = L[i-1,j-1] + 1
      else
        L[i,j] = min (L[i,j-1], L[i-1,j]) + 1

  return L[k,h]
```

La complessità è visto che ci sono due cicli annidati, ripetuti k e h volte, con corpo avente costo costante, $\Theta(hk)$.

Esercizio 2 (9 punti) Per $n > 0$, siano dati due vettori a componenti intere $\mathbf{a}, \mathbf{b} \in \mathbf{Z}^n$. Si consideri la quantità $c(i, j)$, con $0 \leq i \leq j \leq n - 1$, definita come segue:

$$c(i, j) = \begin{cases} a_i & \text{se } 0 < i \leq n - 1 \text{ e } j = n - 1, \\ b_j & \text{se } i = 0 \text{ e } 0 \leq j \leq n - 1, \\ c(i - 1, j - 1) + c(i, j + 1) & 0 < i \leq j < n - 1. \end{cases}$$

Si vuole calcolare la quantità $m = \min\{c(i, j) : 0 \leq i \leq j \leq n - 1\}$.

1. Si fornisca il codice di un algoritmo iterativo bottom-up per il calcolo di m .
 2. Si valuti la complessità esatta dell'algoritmo, associando costo unitario alle somme tra numeri interi (escluse quelle tra indici) e costo nullo a tutte le altre operazioni.
1. Date le dipendenze tra gli indici nella ricorrenza, un modo corretto di riempire la tabella è attraverso una scansione in cui calcoliamo gli elementi in ordine crescente di indice di riga e, per ogni riga, in ordine decrescente di indice di colonna. Il codice è il seguente.

```

COMPUTE(a,b)
n <- length(a)
m = +infinito
for i=1 to n-1 do
  C[i,n-1] <- a_i
  m <- MIN(m,C[i,n-1])
for j=0 to n-1 do
  C[0,j] <- b_j
  m <- MIN(m,C[0,j])
for i=1 to n-2 do
  for j=n-2 downto i do
    C[i,j] <- C[i-1,j-1] + C[i,j+1]
    m <- MIN(m,C[i,j])
return m

```

2.

$$T(n) = \sum_{i=1}^{n-2} \sum_{j=i}^{n-2} 1 = \sum_{i=1}^{n-2} n - 1 - i = \sum_{k=1}^{n-2} k = (n-1)(n-2)/2.$$

Esercizio 2 (9 punti) Sia $n > 0$ un intero. Si consideri la seguente ricorrenza $M(i, j)$ definita su tutte le coppie (i, j) con $1 \leq i \leq j \leq n$:

$$M(i, j) = \begin{cases} 1 & \text{se } i = j, \\ 2 & \text{se } j = i + 1, \\ M(i + 1, j - 1) \cdot M(i + 1, j) \cdot M(i, j - 1) & \text{se } j > i + 1. \end{cases}$$

1. Scrivere una coppia di algoritmi $\text{INIT_M}(n)$ e $\text{REC_M}(i, j)$ per il calcolo memoizzato di $M(1, n)$.
2. Calcolare il numero esatto $T(n)$ di moltiplicazioni tra interi eseguite per il calcolo di $M(1, n)$.

```

INIT_M(n)
if n=1 then return 1
if n=2 then return 2
for i=1 to n-1 do
    M[i,i] = 1
    M[i,i+1] = 2
M[n,n] = 1
for i=1 to n-2 do
    for j=i+2 to n do
        M[i,j] = 0
return REC_M(1,n)

REC_M(i,j)
if M[i,j] = 0 then
    M[i,j] = REC_M(i+1,j-1) * REC_M(i+1,j) * REC_M(i,j-1)
return M[i,j]

```

2.

$$T(n) = \sum_{i=1}^{n-2} \sum_{j=i+2}^n 2 = 2 \sum_{i=1}^{n-2} n - i - 1 = 2 \sum_{k=1}^{n-2} k = (n-2)(n-1)$$

Esercizio 2 (9 punti) Data una stringa $X = x_1, x_2, \dots, x_n$, si consideri la seguente quantità $\ell(i, j)$, definita per $1 \leq i \leq j \leq n$:

$$\ell(i, j) = \begin{cases} 1 & \text{se } i = j \\ 2 & \text{se } i = j - 1 \\ 2 + \ell(i + 1, j - 1) & \text{se } (i < j - 1) \text{ e } (x_i = x_j) \\ \sum_{k=i}^{j-1} (\ell(i, k) + \ell(k + 1, j)) & \text{se } (i < j - 1) \text{ e } (x_i \neq x_j). \end{cases}$$

1. Scrivere una coppia di algoritmi $\text{INIT_L}(X)$ e $\text{REC_L}(X, i, j)$ per il calcolo memoizzato di $\ell(1, n)$.
2. Si determini la complessità *al caso migliore* $T_{\text{best}}(n)$, supponendo che le uniche operazioni di costo unitario e non nullo siano i confronti tra caratteri.

Soluzione:

1. Pseudocodice:

```
INIT_L(X)
  n <- length(X)
  if n = 1 then return 1
  if n = 2 then return 2
  for i=1 to n-1 do
    L[i,i] <- 1
    L[i,i+1] <- 2
  L[n,n] <- 1
  for i=1 to n-2 do
    for j=i+2 to n do
      L[i,j] <- 0
  return REC_L(X,1,n)

REC_L(X,i,j)
  if L[i,j] = 0 then
    if x_i = x_j then L[i,j] <- 2 + REC_L(X,i+1,j-1)
    else for k=i to j-1 do
      L[i,j] <- L[i,j] + REC_L(X,i,k) + REC_L(X,k+1,j)
  return L[i,j]
```

2. Il caso migliore è chiaramente quello in cui tutti i caratteri sono uguali, poiché l'albero della ricorsione in quel caso è unario e i suoi nodi interni corrispondono alle chiamate con indici

$(1, n), (2, n - 1), \dots, (k, n - k + 1)$, ognuno associato a un costo unitario. Ci sono al più $\lfloor n/2 \rfloor$ di queste chiamate, e quindi $T_{\text{best}}(n) = O(n)$.

Esercizio 2 (9 punti) Data una stringa di numeri interi $A = (a_1, a_2, \dots, a_n)$, si consideri la seguente ricorrenza $z(i, j)$ definita per ogni coppia di valori (i, j) con $1 \leq i, j \leq n$:

$$z(i, j) = \begin{cases} a_j & \text{if } i = 1, 1 \leq j \leq n, \\ a_{n+1-i} & \text{if } j = n, 1 < i \leq n, \\ z(i-1, j) \cdot z(i, j+1) \cdot z(i-1, j+1) & \text{altrimenti.} \end{cases}$$

1. Si fornisca il codice di un algoritmo iterativo bottom-up $Z(A)$ che, data in input la stringa A restituisca in uscita il valore $z(n, 1)$.
2. Si valuti il numero esatto $T_Z(n)$ di moltiplicazioni tra interi eseguite dall'algoritmo sviluppato al punto (1).

Soluzione:

1. Date le dipendenze tra gli indici nella ricorrenza, un modo corretto di riempire la tabella è attraverso una scansione “reverse column-major”, in cui calcoliamo gli elementi della tabella in ordine decrescente di indice di colonna e, all'interno della stessa colonna, in ordine crescente di indice di riga. Il codice è il seguente.

```

Z(A)
n = length(A)
for i=1 to n do
    z[1,i] = a_i
    z[i,n] = a_{n+1-i}
for j=n-1 downto 1 do
    for i=2 to n do
        z[i,j] = z[i-1,j] * z[i,j+1] * z[i-1,j+1]
return z[n,1]

```

Si osservi che un altro modo corretto di riempire la tabella è attraverso una scansione “reverse diagonal”, che scansiona per diagonal parallele alla diagonale principale partendo da quella contenente solo $z[1, n]$.

2. Ogni iterazione del doppio ciclo dell'algoritmo esegue due moltiplicazioni tra interi, e quindi

$$\begin{aligned}
 T_Z(n) &= \sum_{j=1}^{n-1} \sum_{i=2}^n 2 \\
 &= \sum_{j=1}^{n-1} 2(n-1) \\
 &= 2(n-1)^2.
 \end{aligned}$$

Equivalentemente, basta osservare che l'algoritmo esegue due moltiplicazioni per ogni elemento di una tabella $(n-1) \times (n-1)$.

Domanda B (6 punti) Si calcoli la lunghezza della longest common subsequence (LCS) tra le stringhe *armo* e *oro*, calcolando tutta la tabella $L[i, j]$ delle lunghezze delle LCS sui prefissi usando l'algoritmo visto in classe.

Soluzione: Si ottiene

	<i>o</i>	<i>r</i>	<i>o</i>	
	0	0	0	0
<i>a</i>	0	0	0	0
<i>r</i>	0	0	1	1
<i>m</i>	0	0	1	1
<i>o</i>	0	1	1	2

Esercizio 2 (9 punti) Sia n un intero positivo. Si consideri la seguente ricorrenza $M(i, j)$ definita su tutte le coppie (i, j) con $1 \leq i \leq j \leq n$:

$$M(i, j) = \begin{cases} 2 & \text{se } i = j, \\ 3 & \text{se } j = i + 1, \\ M(i + 1, j - 1) \cdot M(i + 1, j) + M(i, j - 1) & \text{se } j > i + 1. \end{cases}$$

- (a) Si scriva una coppia di algoritmi $\text{INIT_M}(n)$ e $\text{REC_M}(i, j)$ per il calcolo memoizzato di $M(1, n)$.
 (b) Si calcoli il numero esatto $T(n)$ di moltiplicazioni tra interi eseguite per il calcolo di $M(1, n)$.

Soluzione:

(a) $\text{INIT_M}(n)$

```

if n=1 then return 2
if n=2 then return 3
for i=1 to n-1 do
  M[i,i] = 2
  M[i,i+1] = 3
M[n,n] = 2
for i=1 to n-2 do
  for j=i+2 to n do
    M[i,j] = 0
return REC_M(1,n)
  
```

$\text{REC_M}(i, j)$

```

if M[i,j] = 0 then
  M[i,j] = REC_M(i+1,j-1) * REC_M(i+1,j) + REC_M(i,j-1)
return M[i,j]
  
```

(b)

$$T(n) = \sum_{i=1}^{n-2} \sum_{j=i+2}^n 1 = \sum_{i=1}^{n-2} (n - i - 1) = \sum_{k=1}^{n-2} k = (n-2)(n-1)/2$$

Esercizio 2 (11 punti) Una *longest common substring* di due stringhe X e Y è una sottostringa di X e di Y di lunghezza massima. Si vuole progettare un algoritmo efficiente per calcolare la lunghezza di una longest common substring. Per semplicità si assuma che entrambe le stringhe di input abbiano stessa lunghezza n .

- (a) Qual è la complessità dell'algoritmo esaustivo che analizza tutte le possibili sottostringhe comuni?
 (b) Assumendo di conoscere un algoritmo che determina se una stringa di m caratteri è sottostringa di un'altra stringa di n caratteri in tempo $O(m + n)$, come si può modificare l'algoritmo del punto precedente per renderlo più efficiente?
 (c) Progettare un algoritmo di programmazione dinamica più efficiente di quello del punto precedente. Sono richiesti relazione di ricorrenza sulle lunghezze (senza dimostrazione) e algoritmo bottom-up. (Suggerimento: considerare la lunghezza della longest common substring dei prefissi $X_i = \langle x_1, \dots, x_i \rangle$ e $Y_j = \langle y_1, \dots, y_j \rangle$ che termina con x_i e y_j , rispettivamente.)

Esercizio 2 - Longest Common Substring (LCS)

Problema

Data due stringhe X e Y, una **longest common substring** è una sottostringa comune di X e Y di lunghezza massima.

Nota: Sottostringa significa sequenza **contigua** di caratteri (diverso da subsequence).

Per semplicità: $|X| = |Y| = n$

(a) Complessità Algoritmo Esaustivo

Approccio Esaustivo

L'algoritmo esaustivo enumera **tutte le possibili sottostringhe** di X e verifica se ciascuna è contenuta in Y.

Analisi

Numero di sottostringhe di X:

- Una sottostringa è identificata da una coppia (i, j) con $1 \leq i \leq j \leq n$
- Numero totale: $\binom{n+1}{2} = \frac{n(n+1)}{2} = \Theta(n^2)$

Verifica se una sottostringa è in Y:

- Per ogni sottostringa di lunghezza k, dobbiamo confrontarla con tutte le sottostringhe di Y di lunghezza k
- Numero di sottostringhe di Y di lunghezza k: $n - k + 1$
- Costo di confronto tra due stringhe di lunghezza k: $\Theta(k)$

Costo totale nel caso peggiore:

Per ogni lunghezza k da 1 a n:

- Numero di sottostringhe di X di lunghezza k: $n - k + 1$
- Per ciascuna, confronto con $(n - k + 1)$ sottostringhe di Y
- Costo per confronto: $\Theta(k)$

$$T(n) = \sum_{k=1}^n (n - k + 1) \cdot (n - k + 1) \cdot k = \sum_{k=1}^n (n - k + 1)^2 \cdot k$$

Sostituendo $m = n - k + 1$:

$$T(n) = \sum_{m=1}^n m^2 \cdot (n - m + 1) = \Theta(n^4)$$

Complessità algoritmo esaustivo: $\Theta(n^4)$

(b) Modifica con Oracolo di Substring Checking

Nuovo Scenario

Disponiamo di un algoritmo che verifica se una stringa di m caratteri è sottostringa di una stringa di n caratteri in tempo $O(m + n)$.

Algoritmo Modificato

```
EXHAUSTIVE_LCS_WITH_ORACLE(X, Y, n)
    max_length = 0
    lcs = ""

    for i = 1 to n
        for j = i to n
            substring = X[i..j]
            k = j - i + 1

            if IS_SUBSTRING(substring, Y, k, n)    // O(k + n)
                if k > max_length
                    max_length = k
                    lcs = substring

    return lcs
```

Analisi Complessità Modificata

Numero di sottostringhe: $\Theta(n^2)$

Costo per verifica con oracolo: $O(k + n)$ dove k è la lunghezza della sottostringa

Costo totale:

$$\begin{aligned} T(n) &= \sum_{i=1}^n \sum_{j=i}^n O((j - i + 1) + n) \\ &= \sum_{i=1}^n \sum_{j=i}^n O(j - i + 1) + \sum_{i=1}^n \sum_{j=i}^n O(n) \end{aligned}$$

Il primo termine:

$$\sum_{i=1}^n \sum_{k=1}^{n-i+1} O(k) = \sum_{i=1}^n O((n - i + 1)^2) = O(n^3)$$

Il secondo termine:

$$\sum_{i=1}^n (n - i + 1) \cdot O(n) = O(n) \sum_{i=1}^n (n - i + 1) = O(n) \cdot O(n^2) = O(n^3)$$

Complessità con oracolo: $O(n^3)$

Questo è un miglioramento rispetto a $\Theta(n^4)$, ma ancora non ottimale.

(c) Algoritmo di Programmazione Dinamica

Relazione di Ricorrenza

Definiamo la **tabella di DP**:

$c[i, j]$ = lunghezza della longest common substring che **termina** con x_i e y_j

Ovvero, la più lunga sottostringa comune tale che l'ultimo carattere in X è x_i e l'ultimo carattere in Y è y_j .

Formula Ricorsiva

$$c[i, j] = \begin{cases} 0 & \text{se } i = 0 \text{ o } j = 0 \\ c[i-1, j-1] + 1 & \text{se } i, j > 0 \text{ e } x_i = y_j \\ 0 & \text{se } i, j > 0 \text{ e } x_i \neq y_j \end{cases}$$

Intuizione:

- Se $x_i = y_j$, possiamo estendere la sottostringa comune che termina in x_{i-1} e y_{j-1}
- Se $x_i \neq y_j$, non c'è sottostringa comune che termina esattamente in (i, j)

Soluzione: $\max\{c[i, j] \mid 1 \leq i, j \leq n\}$

Algoritmo Bottom-Up

```
LCS_SUBSTRING_DP(X, Y, n)
    // Inizializzazione
    for i = 0 to n
        c[i, 0] = 0
    for j = 0 to n
        c[0, j] = 0

    max_length = 0
    end_position = 0    // posizione finale della LCS in X

    // Riempimento tabella
    for i = 1 to n
        for j = 1 to n
            if X[i] == Y[j]
                c[i, j] = c[i-1, j-1] + 1

                if c[i, j] > max_length
                    max_length = c[i, j]
```

```

        end_position = i
    else
        c[i, j] = 0

// Ricostruzione soluzione
start_position = end_position - max_length + 1
return X[start_position..end_position]

```

Complessità

- **Spazio:** $\Theta(n^2)$ per la tabella c
- **Tempo:** $\Theta(n^2)$ per riempire la tabella (doppio ciclo con operazioni $O(1)$)

Complessità finale: $\Theta(n^2)$ tempo e $\Theta(n^2)$ spazio

Ottimizzazione Spazio

Poiché $c[i, j]$ dipende solo da $c[i-1, j-1]$, possiamo usare solo **due righe** della tabella:

Spazio ottimizzato: $\Theta(n)$

```

LCS_SUBSTRING_DP_OPTIMIZED(X, Y, n)
    prev = array[0..n] inizializzato a 0
    curr = array[0..n]

    max_length = 0
    end_position = 0

    for i = 1 to n
        curr[0] = 0
        for j = 1 to n
            if X[i] == Y[j]
                curr[j] = prev[j-1] + 1
                if curr[j] > max_length
                    max_length = curr[j]
                    end_position = i
            else
                curr[j] = 0

        swap(prev, curr) // scambia i ruoli delle righe

    return X[end_position - max_length + 1 .. end_position]

```

Complessità ottimizzata: $\Theta(n^2)$ tempo, $\Theta(n)$ spazio

Esempio

X = "ABABC"

Y = "BABCA"

Tabella c[i, j]:

		B	A	B	C	A
	0	0	0	0	0	0
A	0	0	1	0	0	1
B	0	1	0	2	0	0
A	0	0	2	0	0	1
B	0	1	0	3	0	0
C	0	0	0	0	4	0

max_length = 4 (in posizione c[5, 4])

LCS = "BABC" (X[2..5])

Verifica: "BABC" è sottostringa di Y? Y = "BABCA" → Sì, Y[1..4] = "BABC" ✓

La soluzione di GPT è corretta e ben strutturata. Procedo a riformularla in **forma da esame formale**, mantenendo il rigore matematico e lo stile delle soluzioni ufficiali che hai nel materiale del corso.

Alternativa - LCS

(a) Complessità dell'Algoritmo Esaustivo

Strategia: L'algoritmo esaustivo enumera tutte le sottostringhe di X e verifica, per ciascuna, se è contenuta in Y come sottostringa.

Numero di sottostringhe di X:

Una sottostringa è identificata da una coppia di indici (i, j) con $1 \leq i \leq j \leq n$. Il numero totale di sottostringhe è:

$$\sum_{i=1}^n (n - i + 1) = \frac{n(n+1)}{2} = \Theta(n^2)$$

Costo della verifica:

Per verificare se una sottostringa S di lunghezza k è contenuta in Y:

- Confrontiamo S con tutte le sottostringhe di Y di lunghezza k (ce ne sono $n - k + 1$)

- Ogni confronto richiede $\Theta(k)$ operazioni

Nel caso peggiore ($k = \Theta(n)$), ogni verifica costa $\Theta(n^2)$.

Complessità totale:

Abbiamo $\Theta(n^2)$ sottostringhe da verificare, ciascuna con costo $\Theta(n^2)$:

$$T(n) = \Theta(n^2) \cdot \Theta(n^2) = \Theta(n^4)$$

Risposta: La complessità dell'algoritmo esaustivo è $\Theta(n^4)$.

(b) Algoritmo con Oracolo di Substring Matching

Ipotesi: Disponiamo di un algoritmo che verifica se una stringa di m caratteri è sottostringa di una stringa di n caratteri in tempo $O(m + n)$ (ad esempio, algoritmo KMP).

Algoritmo modificato:

```
EXHAUSTIVE_WITH_MATCHING(X, Y, n)
    max_length = 0

    for i = 1 to n                // O(n)
        for j = i to n            // O(n)
            S = X[i..j]           // sottostringa di lunghezza k = j -
            i + 1                  // O(k + n)
            if IS_SUBSTRING(S, Y) // O(k + n)
                if k > max_length
                    max_length = k

    return max_length
```

Analisi della complessità:

- Numero di sottostringhe enumerate: $\Theta(n^2)$
- Costo per verifica con oracolo: $O(k + n)$ dove $k \leq n$

Per ogni sottostringhe di lunghezza k , il costo è $O(k + n) = O(n)$.

Complessità totale:

$$T(n) = \sum_{i=1}^n \sum_{j=i}^n O((j - i + 1) + n) = \Theta(n^2) \cdot O(n) = O(n^3)$$

Risposta: Utilizzando l'algoritmo di matching in tempo lineare, la complessità diventa $O(n^3)$.

(c) Algoritmo di Programmazione Dinamica

Definizione della Sottostruttura

Definiamo la seguente tabella di programmazione dinamica:

$c[i, j]$ = lunghezza della più lunga sottostringa comune che **termina** con x_i in X e y_j in Y

Relazione di Ricorrenza

$$c[i, j] = \begin{cases} 0 & \text{se } i = 0 \text{ oppure } j = 0 \\ c[i-1, j-1] + 1 & \text{se } i, j > 0 \text{ e } x_i = y_j \\ 0 & \text{se } i, j > 0 \text{ e } x_i \neq y_j \end{cases}$$

Motivazione:

- Se $x_i = y_j$, possiamo estendere di 1 la sottostringa comune che terminava in x_{i-1} e y_{j-1}
- Se $x_i \neq y_j$, non esiste alcuna sottostringa comune che termina esattamente in posizione (i, j) , quindi $c[i, j] = 0$
- La soluzione ottimale è: $\max\{c[i, j] \mid 1 \leq i, j \leq n\}$

Algoritmo Bottom-Up

```
LCS_SUBSTRING(X, Y, n)
    // Inizializzazione
    for i = 0 to n
        c[i, 0] = 0
    for j = 0 to n
        c[0, j] = 0

    max_length = 0

    // Riempimento tabella
    for i = 1 to n
        for j = 1 to n
            if X[i] == Y[j]
                c[i, j] = c[i-1, j-1] + 1
                max_length = max(max_length, c[i, j])
            else
                c[i, j] = 0

    return max_length
```

Correttezza

Invariante: Alla fine dell'iterazione (i, j) , $c[i, j]$ contiene la lunghezza della più lunga sottostringa comune che termina esattamente in x_i e y_j .

Dimostrazione per induzione sulla lunghezza totale $i + j$:

- **Base** ($i = 0$ o $j = 0$): $c[i, j] = 0$ per definizione ✓
- **Passo**: Assumiamo vero per tutte le celle (i', j') con $i' + j' < i + j$
 - Se $x_i = y_j$: per ipotesi induttiva, $c[i-1, j-1]$ è corretto, quindi $c[i, j] = c[i-1, j-1] + 1$ è corretto
 - Se $x_i \neq y_j$: non può esistere sottostringa che termina in entrambe le posizioni, quindi $c[i, j] = 0$ è corretto ✓

Complessità

Tempo:

- Inizializzazione: $\Theta(n)$
- Doppio ciclo: $\Theta(n^2)$ iterazioni, ciascuna con operazioni $O(1)$
- **Totale: $\Theta(n^2)$**

Spazio: Matrice $n \times n \rightarrow \Theta(n^2)$

Ottimizzazione dello spazio: Poiché $c[i, j]$ dipende solo da $c[i-1, j-1]$, possiamo mantenere solo due righe della tabella, riducendo lo spazio a $\Theta(n)$.

Riepilogo Complessità

Metodo	Complessità Temporale	Complessità Spaziale
Esaustivo naive	$\Theta(n^4)$	$O(1)$
Esaustivo con matching lineare	$O(n^3)$	$O(n)$
Programmazione dinamica	$\Theta(n^2)$	$\Theta(n^2)$ [$\Theta(n)$ ottim.]

Risposta finale: L'algoritmo di programmazione dinamica ha complessità $\Theta(n^2)$ tempo e $\Theta(n^2)$ spazio (riducibile a $\Theta(n)$ spazio).

Esercizio 2 (8 punti) Date due stringhe $X = \langle x_1, x_2, \dots, x_m \rangle$ e $Y = \langle y_1, y_2, \dots, y_n \rangle$, si consideri la seguente quantità $\ell(i, j)$, definita per ogni coppia di valori i, j con $0 \leq i \leq m$ e $0 \leq j \leq n$:

$$\ell(i, j) = \begin{cases} 1 & \text{se } i = 0 \text{ o } j = 0 \\ 3\ell(i, j-1) & \text{se } i, j > 0 \text{ e } x_i = y_j \\ 2\ell(i-1, j-1) - \ell(i-1, j) & \text{se } i, j > 0 \text{ e } x_i \neq y_j. \end{cases}$$

Si vuole calcolare la quantità $q = \max\{\ell(i, j) : 0 \leq i \leq m, 0 \leq j \leq n\}$.

- Scrivere un algoritmo bottom-up per il calcolo di q .
- Determinare la complessità *esatta* dell'algoritmo, supponendo che le uniche operazioni di costo unitario e non nullo siano i confronti tra caratteri.

Esercizio 2 - Calcolo di q

Testo

Date due stringhe $X = (x_1, x_2, \dots, x_m)$ e $Y = (y_1, y_2, \dots, y_n)$, si consideri la seguente quantità $\ell(i, j)$, definita per ogni coppia di valori i, j con $0 \leq i \leq m$ e $0 \leq j \leq n$:

$$\ell(i, j) = \begin{cases} 1 & \text{se } i = 0 \text{ o } j = 0 \\ 3\ell(i, j-1) & \text{se } i, j > 0 \text{ e } x_i = y_j \\ 2\ell(i-1, j-1) - \ell(i-1, j) & \text{se } i, j > 0 \text{ e } x_i \neq y_j \end{cases}$$

Si vuole calcolare la quantità $q = \max\{\ell(i, j) : 0 \leq i \leq m, 0 \leq j \leq n\}$.

(a) Algoritmo Bottom-Up per il Calcolo di q

Analisi delle Dipendenze

Per calcolare $\ell[i, j]$, abbiamo bisogno di:

- $\ell[i, j-1]$ (stessa riga, colonna precedente)
- $\ell[i-1, j-1]$ (riga precedente, colonna precedente)
- $\ell[i-1, j]$ (riga precedente, stessa colonna)

Tutte le dipendenze hanno indici strettamente minori di (i, j) .

Ordine di Riempimento

Un ordine corretto è: per righe crescenti (i da 0 a m), e all'interno di ogni riga per colonne crescenti (j da 0 a n).

Dimostrazione della correttezza: Quando calcoliamo $\ell[i, j]$:

- $\ell[i, j-1]$ è stato calcolato nell'iterazione precedente del ciclo interno (colonna $j-1 < j$)
- $\ell[i-1, j-1]$ e $\ell[i-1, j]$ sono stati calcolati nel ciclo esterno precedente (riga $i-1 < i$)

Quindi tutte le dipendenze sono soddisfatte.

Pseudocodice

```
COMPUTE_Q(X, Y, m, n)
    // Inizializzazione riga 0
    for j = 0 to n
         $\ell[0, j] = 1$ 

    // Inizializzazione colonna 0
    for i = 1 to m
         $\ell[i, 0] = 1$ 

    q = 1
```

```
// Riempimento tabella: righe crescenti, colonne crescenti
for i = 1 to m
  for j = 1 to n
    if X[i] == Y[j]
      Q[i, j] = 3 * Q[i, j-1]
    else
      Q[i, j] = 2 * Q[i-1, j-1] - Q[i-1, j]

    if Q[i, j] > q
      q = Q[i, j]

return q
```

(b) Complessità Esatta dell'Algoritmo

Operazioni di costo unitario e non nullo: confronti tra caratteri.

Tutte le altre operazioni (assegnamenti, operazioni aritmetiche su interi, confronti di interi) non vengono conteggiate secondo la richiesta.

Conteggio dei Confronti tra Caratteri

Inizializzazione: nessun confronto tra caratteri.

Doppio ciclo:

- Numero di iterazioni: $m \times n$
- Per ogni iterazione: esattamente 1 confronto tra caratteri ($X[i] == Y[j]$)

Complessità Esatta

$$T(m, n) = mn$$

Risposta: La complessità esatta, considerando solo i confronti tra caratteri, è $\Theta(mn)$.